Techniques d'indexation

Khaled Jouini
j.khaled@gmail.com

Institut Supérieur d'Informatique et des Technologies de Communication

—–

La reproduction de ce document par tout moyen que ce soit, sans l'avis de l'auteur, est interdite conformément aux lois protégeant la propriété intellectuelle

Ce document peut comporter des erreurs, à utiliser donc en connaissance de cause!

Syllabus du cours

Pré-requis

- Techniques d'indexation de base (Hachage, Arbre B+, Arbre R, etc.)
- Architecture des ordinateurs (hiérarchie des mémoires, etc.)
- Informatique répartie (réseaux pair-à-pair, BDs réparties, etc.) - Connaissances utiles, mais pas indispensables

Objectifs

- Connaître les fondements et les concepts clés de l'indexation
- Prendre conscience des apports et des problèmes posés par l'indexation

Syllabus du cours

Pour en savoir plus...

- *The Indispensability of Dispensable Indexes*. E. Bertino, B.C. Ooi. IEEE Transactions on Knowledge and Data Engineering. V. 11 Issue 1. 1999
- Documentation Oracle : http://www.oracle.com/technetwork/documentation/.
- *Advanced Database Indexing*. Y. Manolopouos, Y.Theodoridis, V.J. Tsotras. Kluwer Academic Publisher. 2000.
- *Image databases: Search and Retrieval of Digital Imagery*. V. Castelli, L.D. Bergman. Kindle Edition. 2002.
- *Chord: A Scalable Peer-to-peer Lookup Service for Internet Applications*. I. Stoica. ACM SIGCOMD. 2001.
- *Lazy-Adaptive Tree: An Optimized Index Structure for Flash Devices*. D. Agrawal & Al. VLDB'09, Août 2009, Lyon, France
- *Introduction to spatial databases: with application to GIS* Ph. Rigaux,M. Scholl,A. Voisard, 2006
- Etc.

Chapitre 1 - Introduction générale

- Qu'est ce que l'indexation?
- L'indexation est elle indispensable?
- Choix de la "bonne" structure d'indexation
- Plan du cours

Qu'est ce que l'indexation?

- Notion d'index : la même que pour un livre.
- Exemple : trouver toutes les pages où apparaît le mot Sousse/ trouver tous les clients résidant à Sousse

Livre	Informatique (Ex. Base de Données)
Mots-clés de l'index	Valeurs d'un attribut
Pages	Adresses mémoires où sont stockées ces valeurs

- Intérêt de l'**indexation** : permettre une **recherche rapide** d'une information.

Chapitre 1 - Introduction générale

- Qu'est ce que l'indexation?
- L'indexation est elle indispensable?
- Choix de la "bonne" structure d'indexation
- Plan du cours

L'indexation est elle indispensable?

Recherche de l'information sans indexation

- Parcours séquentiel (**complexité linéaire**)
- Recherche par dichotomie si les données sont triées (**complexité logarithmique**)

⇒ Tolérable pour des "petits" espaces de recherche, mais pas pour les grands volumes d'informations.
Ex. BD d'empreintes digitales, Web, data warehouse, etc.

L'indexation est elle indispensable?

Recherche de l'information avec un index

- Parcours de l'index puis ***accès direct*** à l'information (enregistrement)
- Mais,
  - Temps additionnel de maintenance de l'index suite aux opérations de modification (ajout/suppresion/mise-à-jour des données)
  - Espace supplémentaire pour le stockage de l'index

⇒ **Index** : structure auxiliaire qui devient indispensable lors que

1. L'espace de recherche de l'information est "grand"
2. Les opérations de recherche prédominent les opérations de modification

Chapitre 1 - Introduction générale

- Qu'est ce que l'indexation?
- L'indexation est elle indispensable?
- Choix de la "bonne" structure d'indexation
- Plan du cours

Choix de la "bonne" structures d'indexation

- Il existe des centaines de méthodes d'indexation, mais aucune n'est universelle
- Le choix de la "bonne" structure dépend de plusieurs paramètres :
  1. **Caractéristiques des données** : données relationnelles, multimédias, spatiales, temporelles, etc.
  2. **Type de recherches fréquentes** : recherche exacte, par intervalle, etc.
  3. **Support de stockage** : données résidant sur disque dur, mémoire vive, mémoire flash (*Solid State Drive*), etc.
  4. **Architecture de la machine** : mono-core, multi-cores et.
  5. **Architecture du réseau** : centralisé, réparti, et.
  6. Etc.

Chapitre 1 - Introduction générale

- Qu'est ce que l'indexation?
- L'indexation est elle indispensable?
- Choix de la "bonne" structure d'indexation
- Plan du cours

Plan du cours

1 **Introduction générale**
- Qu'est ce que l'indexation?
- L'indexation est elle indispensable?
- Choix de la "bonne" structure d'indexation
- Plan du cours

2 **Structures d'indexation fondamentales**
- Introduction
- I. Structures d'indexation mono-attribut
  - 1. Index non-dense (Sparse)
  - 2. Index dense
  - 3. Index multi-niveaux
  - 4. Arbre B+
  - 5. (Table de) Hachage statique
  - 6. (Table de) Hachage extensible
  - 7. Index bitmap
  - Filtres de bloom

3 **II. Structures d'indexation textuelle en mémoire**
- 1. TRIE : Arbre de préfixes
- 2. FST
- 3. Exercice

Chapitre 2 - Structures d'indexation fondamentales

- Introduction
- I. Structures d'indexation mono-attribut

Introduction

Définition (Clé de recherche)

Liste des attributs sur lesquels les données sont indexées

Types de recherche dans un index

- **Recherche par clé** (recherche exacte)
  - A partir d'une valeur de la clé de recherche, trouver tous les enregistrements possédant cette valeur
  - Ex. "Trouver tous les employés ayant un salaire égal à 1000"
- **Recherche par intervalle**
  - Trouver tous les enregistrements dont la valeur d'un attribut est dans un intervalle donné
  - Ex. "Trouver tous les employés ayant un salaire entre 500 et 1000"

Introduction

Types d'index

- **Index mono-attribut** : clé de recherche composée d'un seul attribut

  Ex. "Trouver tous les employés ayant un salaire égal à 1000"

- **Index multi-attributs** : clé de recherche composée par plusieurs attributs

  Ex. "Trouver tous les employés ayant un salaire égal à 1000 et habitant la ville de Sousse"

Ce chapitre...

1. Structures d'indexation mono-attribut
2. Structures d'indexation multi-attributs

Chapitre 2 - Structures d'indexation fondamentales

- Introduction
- I. Structures d'indexation mono-attribut
  - 1. Index non-dense (Sparse)
  - 2. Index dense
  - 3. Index multi-niveaux
  - 4. Arbre B+
  - 5. (Table de) Hachage statique
  - 6. (Table de) Hachage extensible
  - 7. Index bitmap
  - Filtres de bloom

1. Index non-dense (*Sparse*)

Index non-dense (Sparse)

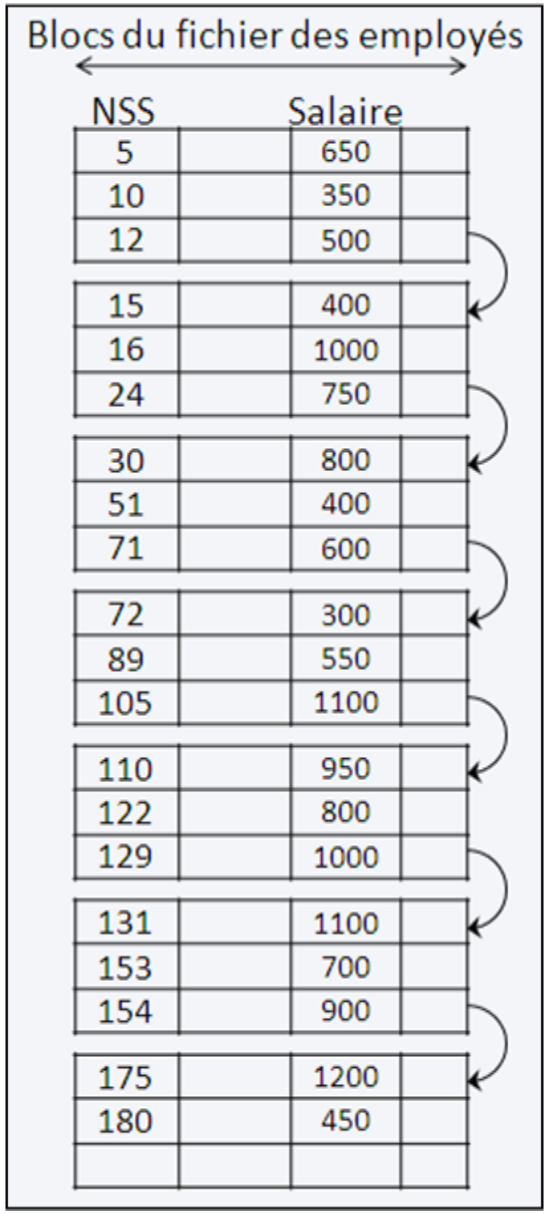
- S'applique aux fichiers de données triés.

  Exemple : fichier des employés trié sur le numéro de sécurité sociale (NSS)

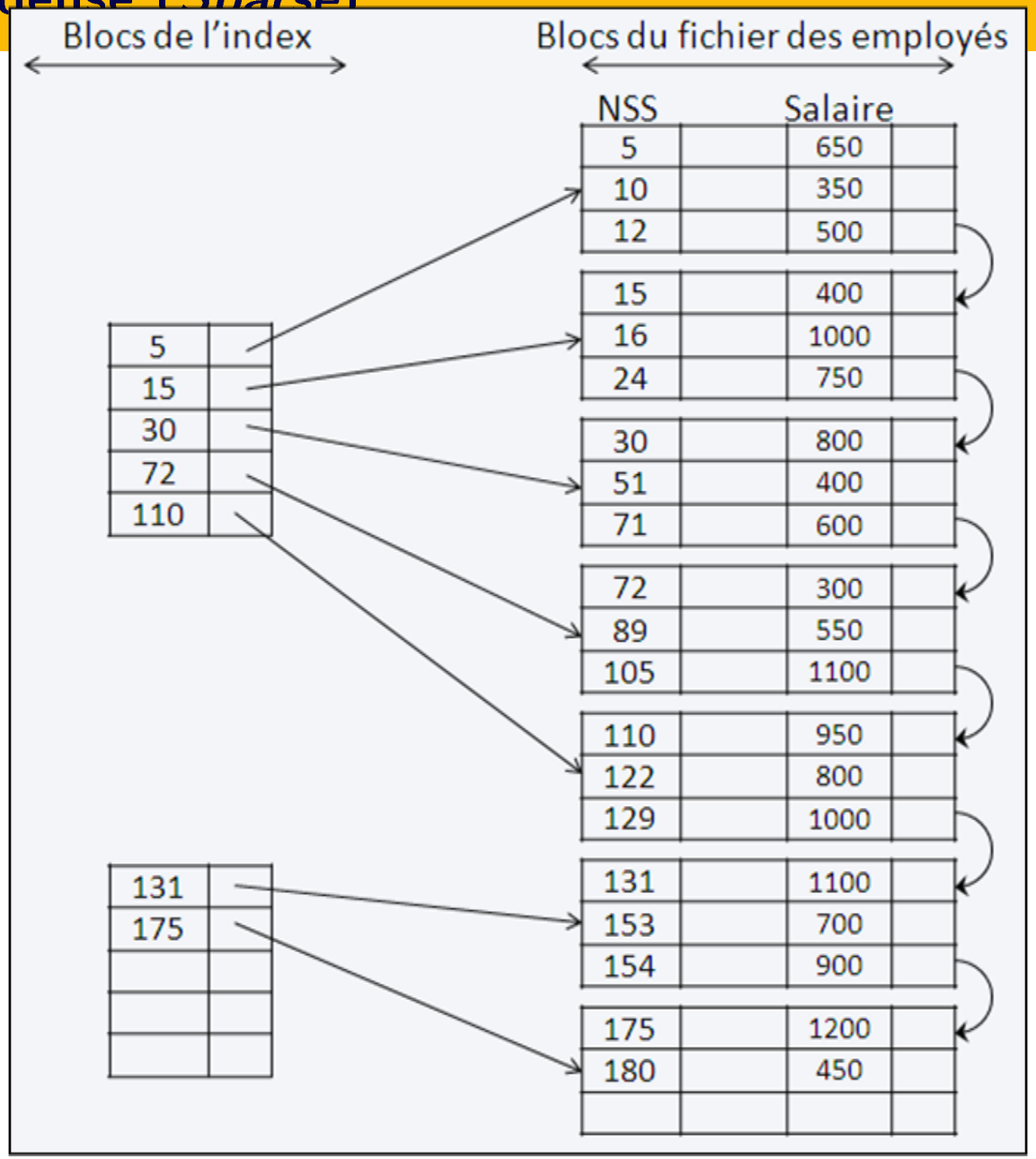
- Construction de l'index :
  - Clé de recherche : attribut selon lequel le fichier est trié

    Exemple : NSS.
  - Principe : associer (dans l'index) l'adresse de chaque bloc du fichier de données à la valeur de la première clé du bloc (la plus petite valeur de la clé apparaissant dans le bloc)

1. Index non-dense (*Sparse*)



1. Index non-dense (*Sparse*)



1. Index non-dense (*Sparse*)

Recherche d'une clé k

1. Recherche dans l'index la plus grande clé strictement inférieure à k
2. Suivre l'adresse correspondante

Recherche par intervalle $[k_{min}, k_{max}]$

1. Recherche dans l'index la plus grande clé strictement inférieure à k_{min}
2. Suivre l'adresse *adr* correspondante
3. Parcourir séquentiellement le fichier de données à partir du bloc *adr*
4. S'arrêter lorsque une clé $> k_{max}$ est rencontrée

2. Index dense

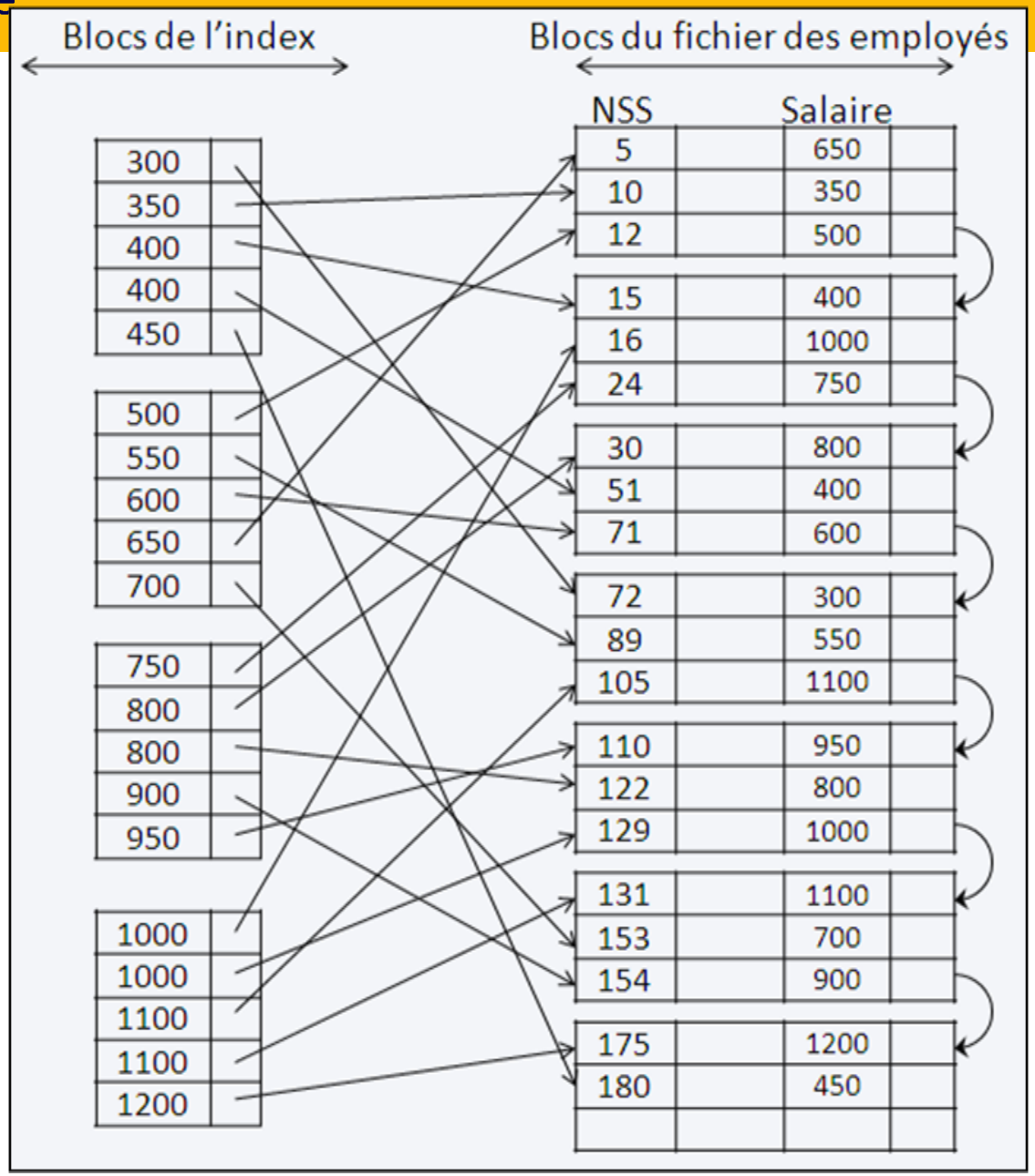
Index dense

- S'applique aux fichiers de données non triés ou triés sur un attribut différent de la clé de recherche
- Construction de l'index :
  - Associer (dans l'index) chaque valeur de la clé au bloc de données où l'enregistrement correspondant apparaît

  $\Rightarrow$ toutes les valeurs doivent apparaître dans l'index (d'où le nom dense)

  - Combien d'index denses et d'index non dense peut on construire sur un même fichier?

2. Index dense



3. Index multi-niveaux - a. Principe

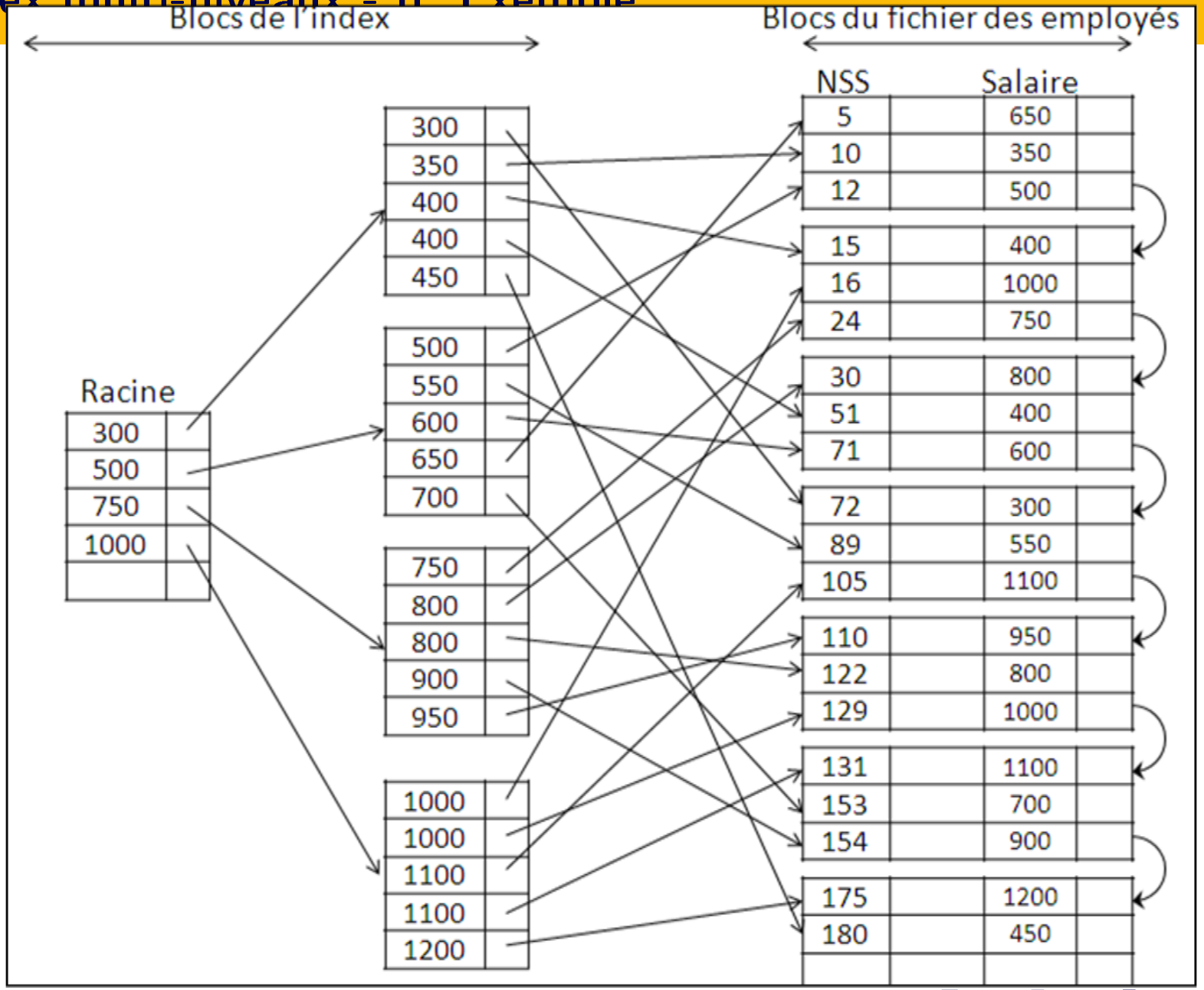
Problème : lorsque l'index devient à son tour volumineux, les recherches sont à nouveau pénalisées

Solution :

1. Indexer les blocs de l'index. Un deuxième **niveau** d'indexation est obtenu
2. Répéter l'opération jusqu'à ce qu'il n'y ait plus qu'un seul bloc au plus haut niveau de l'index

Structure obtenue : **fichier séquentiel indexé** (*Indexed Sequential File*) ou structure **ISAM** (*Indexed Sequential Access Method*)

3. Index multi-niveaux - b. Exemple



3. Index multi-niveaux - c. Niveaux et noeuds

- **Index multi-niveaux** : chaque niveau partitionne davantage l'espace de recherche
- Trois types de noeud (bloc)
  - **Racine** : noeud du plus haut niveau de l'index
  - **Feuilles** (ou **terminaux**): noeuds se trouvant au plus bas niveau de l'index. Contiennent les adresses des blocs de données
  - **Internes** : noeuds se trouvant entre la racine et les feuilles
- **Noeuds non terminaux** : noeuds internes + racine (si plus de 2 niveaux)
- L'index est **équilibré** (*Balanced*) : toutes les feuilles de l'index sont à la même distance de la racine.
- Dans la pratique l'index ne dépasse pas trois niveaux et la racine et les noeuds de second niveau sont gardés en mémoire vive

3. Index multi-niveaux - d. Performances

Performances

- Paramètres :

Paramètre	Signification
B	Capacité d'un noeud (nombre d'entrées par noeud)
n	Nombre de clés de recherche (ou d'enregistrements)

- **Recherche exacte** : $\mathbf{O}(\log_B(\frac{n}{B}))$
- **Recherche par intervalle** : $\mathbf{O}(\log_B(\frac{n}{B}) + \frac{a}{B})$, avec a le nombre de valeurs appartenant à l'intervalle
- **Coût du stockage** (index non dense) : $\mathbf{O}(\frac{n}{B^2})$
- Exemple concret : n=1 million d'enregistrements, B=100, $\frac{n}{B}$=10000 blocs, $\log_B(\frac{n}{B})$=2, $\frac{n}{B^2}$=100.

$\Rightarrow$ très bonnes performances pour les opérations de recherche et le stockage

3. Index multi-niveaux - e. Limites

Structure adaptée aux fichiers peu évolutifs. En cas de grossissement :

- Un bloc saturé déborde ⇒ un nouveau bloc de débordement est alloué.
- Peut conduire à de longues chaînes de débordement et à une dégradation des performances.
- Peut être évité par des réorganisations (recréations) périodiques de l'index (lourdes si cardinalité élevée)

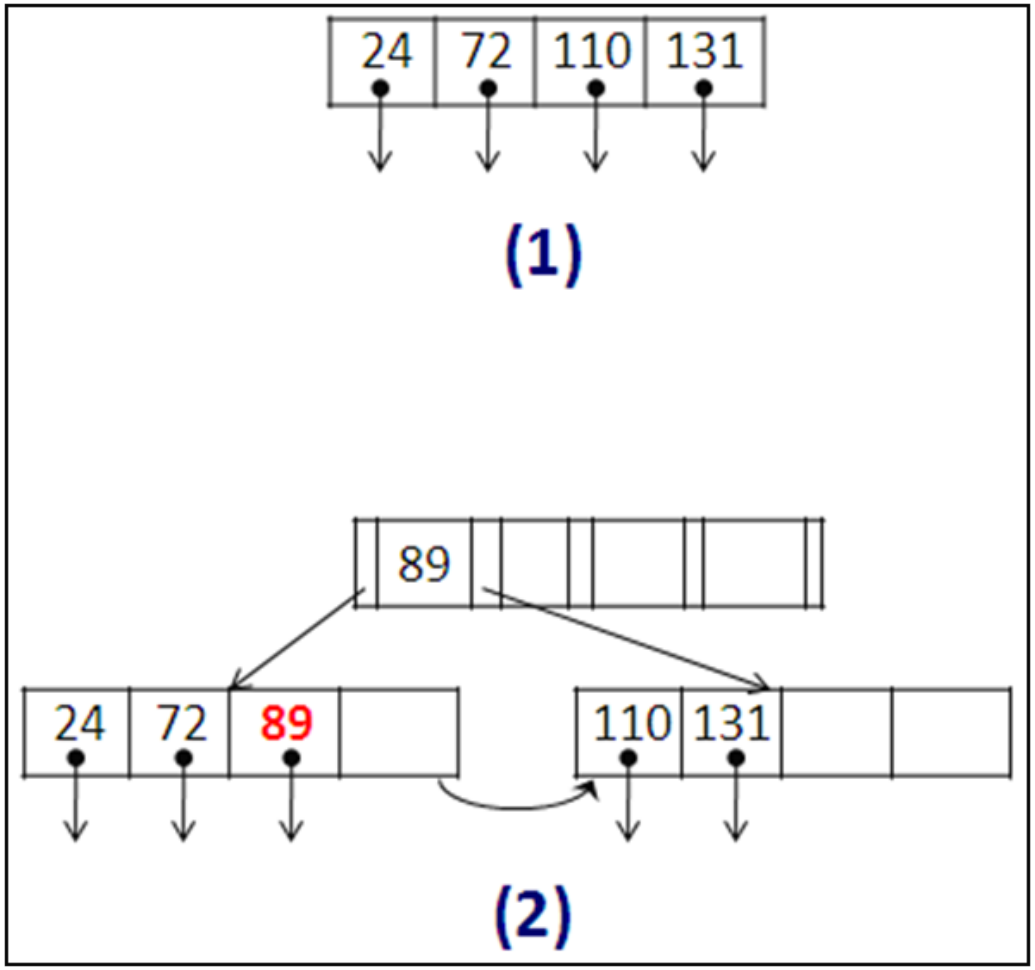
⇒ besoin d'une structure permettant une **réorganisation dynamique** sans dégradation de performances.

4. Arbre B+ - a. Vue d'ensemble

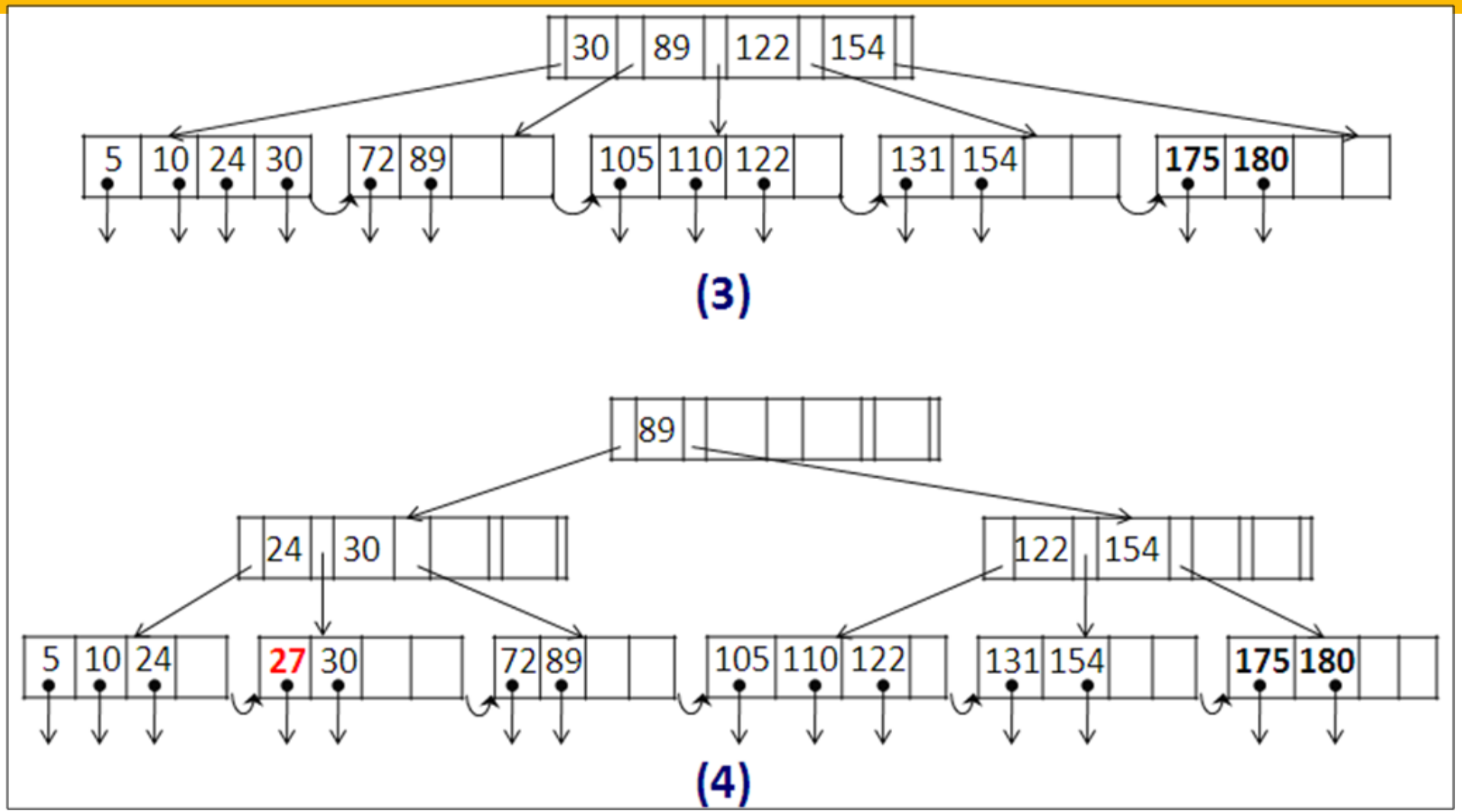
Arbre B+

- Structure d'indexation arborescente, équilibrée (*Balanced*), inspirée de l'indexation des fichiers séquentiels
- **Objectif** : amortissement des opérations de mises-à-jour
- ⇒ Création et maintenance de manière incrémentale et dynamique

4. Arbre B+ - b. Exemple



4. Arbre B+ - b. Exemple



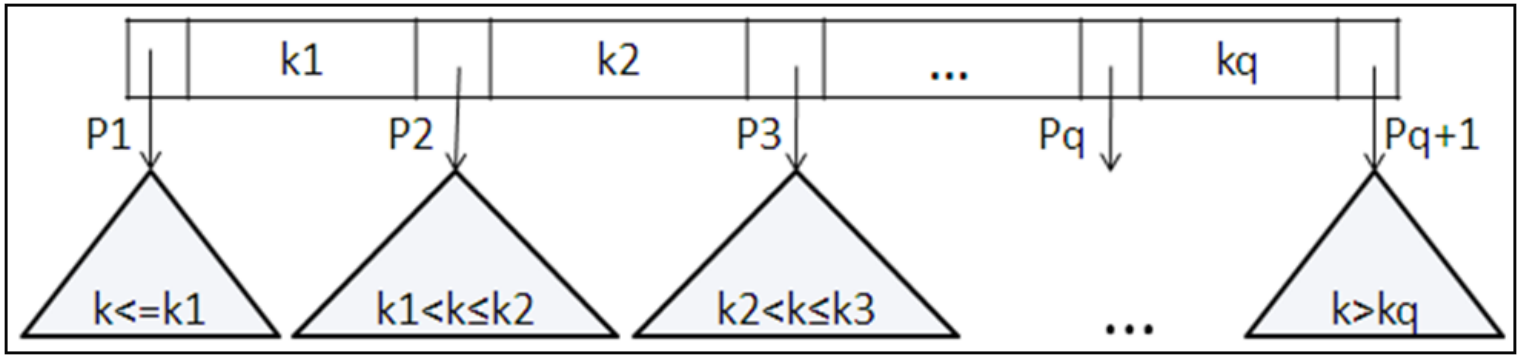
4. Arbre B+ - c. Définition

Un arbre B+ d'**ordre** m est un arbre équilibré tel que :

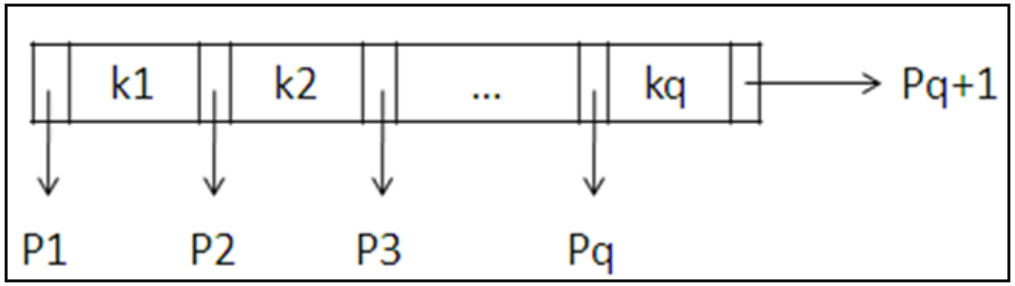
1. Chaque noeud "père" (non terminal) possède au maximum m fils
2. Chaque noeud non terminal, excepté la racine, possède au minimum $\lceil \frac{m}{2} \rceil$ fils
3. La racine a 0 ou au moins deux fils
4. Un noeud non terminal contenant q clés possède $q + 1$ fils
5. Les clés de chaque noeud sont triées
6. Pour optimiser les recherches par intervalle
   1. Les noeuds feuilles sont chaînés
   2. Toutes les clés apparaissent au niveau de feuilles

4. Arbre B+ - d. Structure d'un noeud

Structure d'un noeud non terminal



Structure d'un noeud feuille



4. Arbre B+ - e. Recherche

Recherche d'une clé k

1. Partir de la racine
2. Soit $k_1, k_2, ..., k_q$ les valeurs de clés du noeud courant
   1. Si $k \leq k_1$ (resp. $k > k_q$), la recherche continue dans le noeud référencé par p_1 (resp. p_{q+1})
   2. Sinon il existe $i \in [1, q[$ tel que $k_i < k \leq k_{i+1}$, la recherche continue dans le noeud référencé par p_{i+1}
3. L'étape 2 est répétée jusqu'à ce qu'un noeud feuille est atteint

4. Arbre B+ - e. Recherche

Recherche par intervalle $[k_{min}, k_{max}]$

1. Chercher la feuille contenant k_{min}
2. Suivre le chaînage des feuilles
3. S'arrêter lorsqu'une clé k_i telle que $k_{max} < k_i$ est rencontrée

4. Arbre B+ - f. Insertion

Insertion d'une entrée (paire) [k,adr]

1. Chercher le noeud feuille f où (k,adr) doit être insérée
2. Si suite à l'insertion f déborde, il est éclaté :
   1. Un nouveau noeud f' est créé.
   2. Soit k_{med} la clé médiane de f'. Les entrées de f dont la clé est supérieure à k_{med} sont déplacées vers f' (f et f' s'en trouvent à moitié pleins)
3. L'information sur l'éclatement est envoyée au niveau supérieur de l'arbre
   1. Si f est une racine (n'a pas de père), un nouveau noeud racine est créé. La nouvelle racine contient, l'adresse de f, suivie par l'entrée $[k_{med}, f']$
   2. Si f a un père p l'arbre la paire $[k_{med}, f']$ y est ajoutée. Si p déborde, il est éclaté comme en 2, à l'exception que k_{med} n'est pas dupliqué dans le noeud éclaté

4. Arbre B+ - f. Insertion

Exercice : construisez un Arbre B+ d'ordre 4 avec les valeurs suivantes

1. 35, 10, 15, 20, 50, 30, 45, 55, 60, 40, 25, 5
2. 5, 10, 15, 20, 25, 30, 35, 40, 45, 50, 55, 60

4. Arbre B+ - g. Performances

- **Paramètres :**

Paramètre	Signification
B	Capacité d'un bloc en terme de nombre total d'entrées qu'il peut contenir
b	Facteur de ramification (*fanout*) : nombre moyen d'entrées stockées dans un noeud. Estimée par $0.67 \times B$
n	Nombre de clés de recherche (ou d'enregistrements)

- **Recherche exacte** :
- **Recherche par intervalle** : $\mathbf{O}(\log_b(\frac{n}{b}) + \frac{a}{b})$, avec a le nombre de valeurs appartenant à l'intervalle
- **Coût du stockage** : $\mathbf{O}(\frac{n}{b})$
- **Insertion** :

⇒ (Presque) parfait!

⇒ Utilisé par défaut dans la plupart des SGBDs et les SGF

4. Arbre B+ - h. Arbre B+ plaçant (groupant)

Arbre B+ plaçant (groupant)

- Les feuilles contiennent les enregistrements ⇒ l'index contrôle le placement physique des enregistrements
- **Intérêt** :
  - Recherche par clé : éviter une indirection
  - Recherche par intervalle : maximiser la proximité
- Il ne peut exister qu'un seul index groupant par fichier (table)
- Implanté dans la plupart des SGBDs (Oracle : *Index Organized Table*, etc.)

5. Hachage statique - a. Vue d'ensemble

Hachage

- Concurrent de l'arbre B+
  - Meilleur pour les recherches par clé
  - Occupe très peu de place
- Mais
  - Réorganisation difficile
  - Ne permet pas les recherches par intervalle

5. Hachage statique - b. Principe

Principe

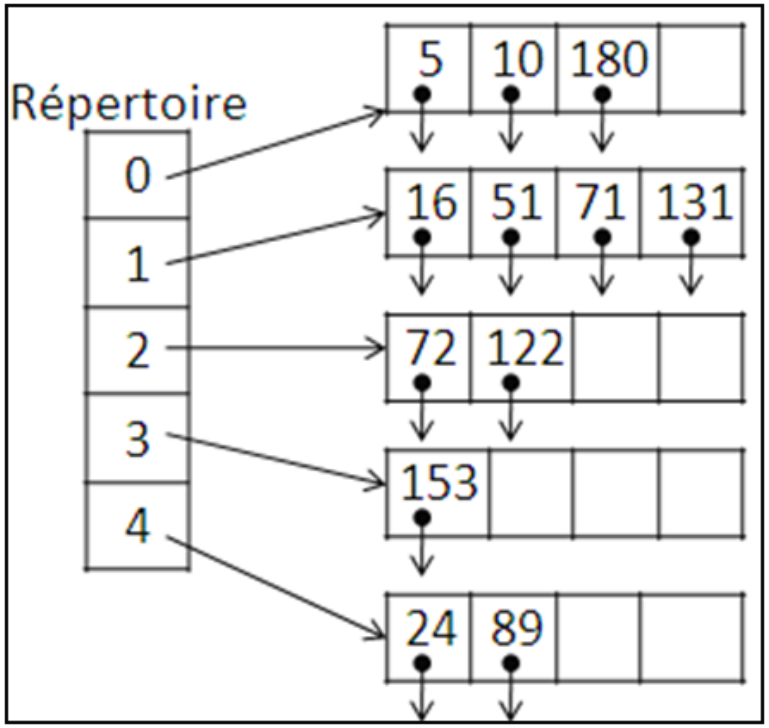
- s blocs sont alloués à la structure ($s \ll n$, avec n le nombre total d'entrées (k, adr))
- Une **fonction de hachage h** associe chaque valeur de clé à un des s blocs. Exemple : $h(k) = k \; mod \; s$
- Recherche d'une clé $k \Rightarrow$ calcul de $h(k)$ pour trouver l'adresse du bloc contenant k
- h est choisie de manière à répartir uniformément les clés dans les blocs

$\Rightarrow$ Simple et efficace !

5. Hachage statique - b. Principe

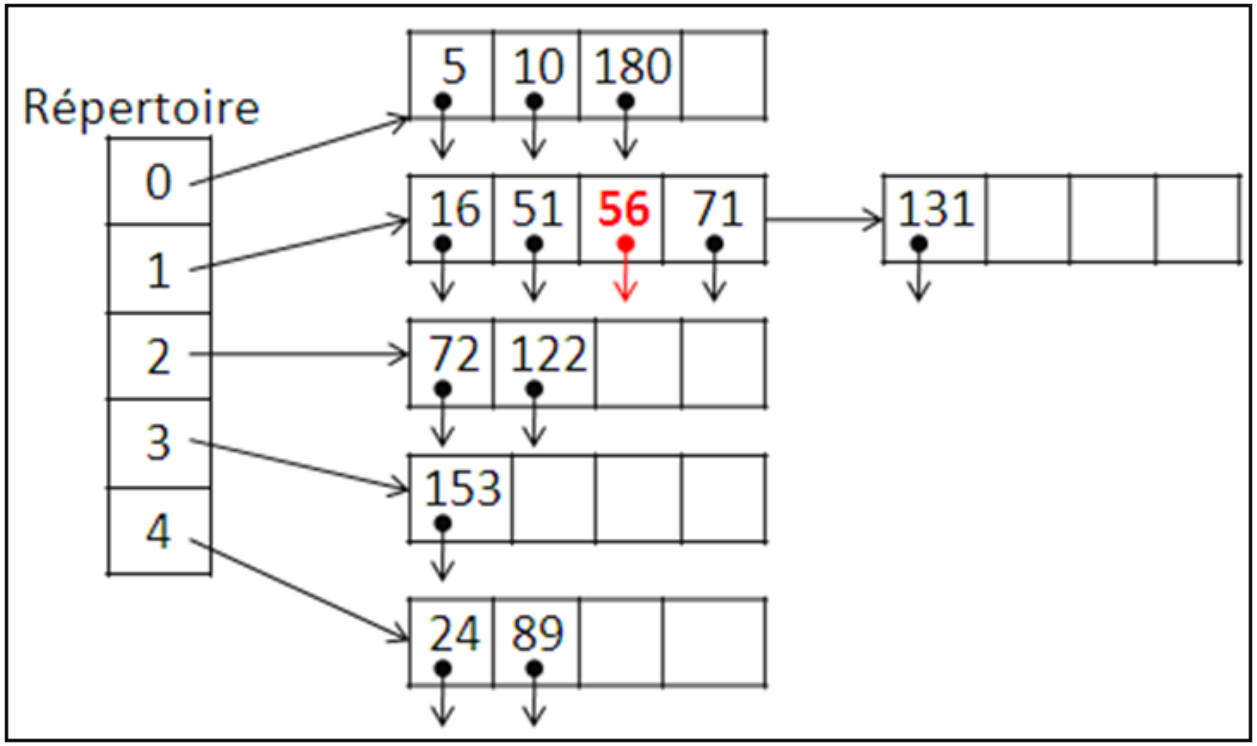
Exemple : $n = 12$, hypothèse : 4 entrées par bloc

- 5 blocs sont alloués (pour garder un peu de marge)
- $h(k) = k \; mod \; 5$
- Un **répertoire** de 5 entrées de 0 à 4 pointe vers les blocs



5. Hachage statique - c. Limites

- Pas de recherche par intervalle
- En cas de débordement : allocation d'un nouveau bloc, chaîné à l'ancien ⇒ dégradation des performances



6. Hachage extensible - a. Vue d'ensemble

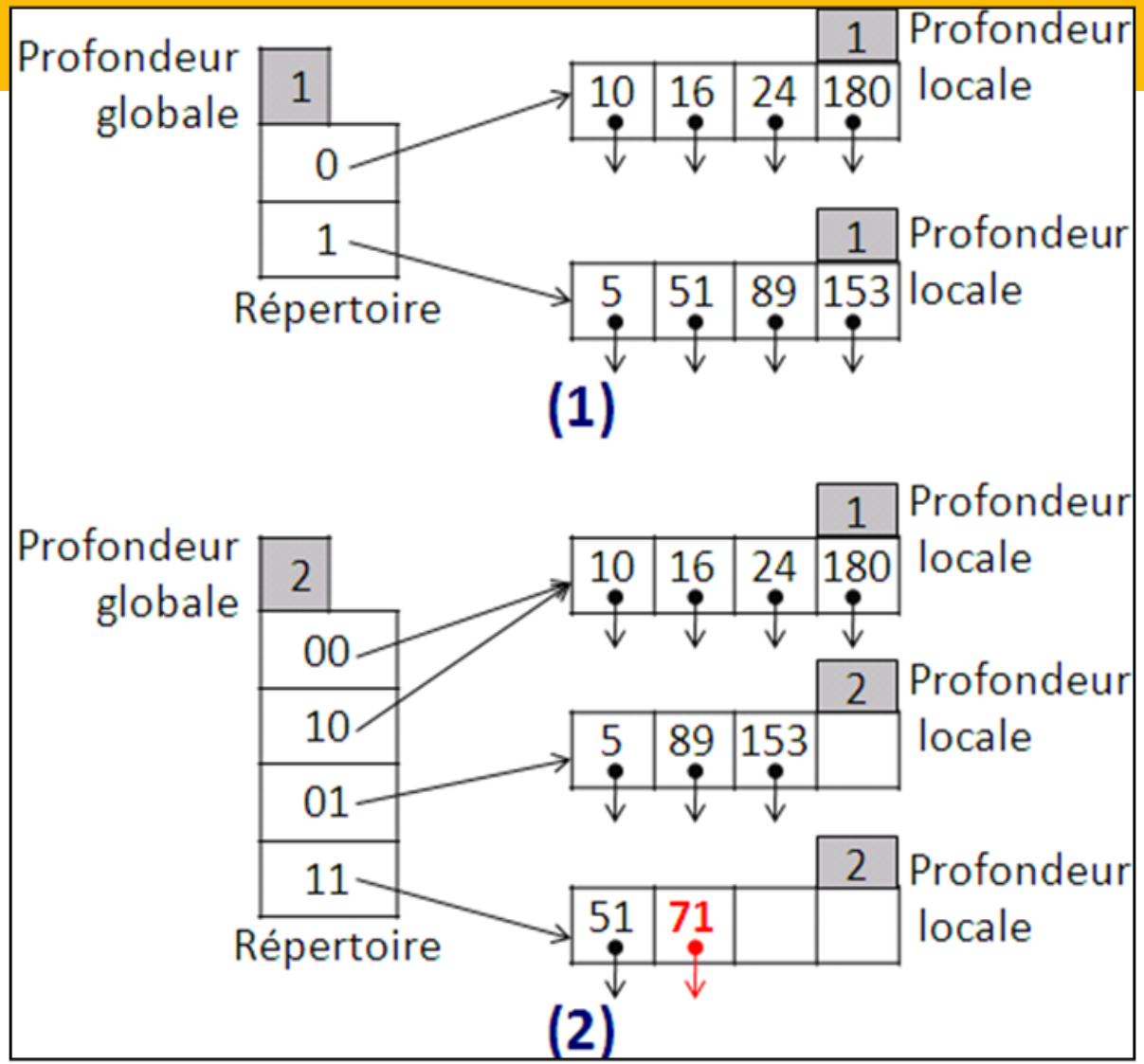
Hachage extensible

- **Objectif** : **réorganiser** la table de hachage **dynamiquement** en fonction des insertions/suppressions
- ***h(k)*** : retourne un code binaire (Ex. $h(5) = 101$)
- Le nombre d'entrées dans le répertoire est variable :
  - Contient 2^d entrées, avec **d** **profondeur globale** du répertoire.
  - Initialement $d = 1$, puis en fonction des éclatements, $d = 2$, $d = 3$, etc.
- Les d derniers bits du résultat de h sont utilisés pour la répartition des valeurs de la clé (initialement 1 seul bit, ensuite 2 bits, etc.)

6. Hachage extensible - a.

NSS	$h(NSS)$
16	10000
153	10011001
24	11000
5	101
10	1010
180	10110100
89	1011001
51	110011
71	**1000111**

- Plusieurs entrées du répertoire peuvent pointer vers le même bloc
- A chaque bloc est associé une profondeur locale (quelle utilité?)



6. Hachage extensible - b. Insertion

Insertion de (k, adr) dans une table de hachage de profondeur d

1. Les d derniers bits de $h(k)$ sont utilisés pour trouver le bloc B où (k, adr) est à insérer.
2. Si B déborde,
   1. Un nouveau bloc B' est alloué
   2. Soit d' la profondeur locale de B. La répartition des entrées entre B et B' se fait selon le $d' - 1$ avant dernier bit : les entrées de B ayant ce bit à 1 sont déplacées vers B'
   3. La profondeur locale de B et B' est incrémentée ($d' \leftarrow d' + 1$)
   4. Si $d' <= d$, le répertoire est mis à jour pour pointer vers B'
   5. Sinon ($d' > d$), la taille du répertoire est doublée ($d \leftarrow d + 1$). Le répertoire est mis à jour en conséquence.

6. Hachage extensible - c. Remarques

- Hachage intéressant lorsque
  - Les données sont peu évolutives
  - Les recherches se font par clé

  Autrement l'Arbre B+ est meilleur

- Dans la plupart des SGBDs seul le hachage statique est implanté (y compris Oracle)

6. Hachage extensible - d. Exercices

1. Construisez une table de hachage statique avec les valeurs ci-dessous en supposant que le nombre de blocs réservés soit 4 et qu'un bloc ne peut contenir au plus que 3 entrées.

   23, 39, 28, 40, 29, 24, 22, 12, 53

6. Hachage extensible - d. Exercices

2. Construisez une table de hachage dynamique avec les valeurs ci-dessous en supposant qu'un bloc ne peut contenir au plus que 3 entrées. Donnez une représentation de la table, chaque fois qu'un nouveau bloc est alloué.

Valeur	*h(Valeur)*
23	10111
39	100111
28	11100
40	101000
29	11101
24	11000
22	10110
12	1100
53	110101

7. Index Bitmap - a. Le problème

Problème : comment indexer des données sur un attribut A dont la cardinalité de son domaine de valeurs est "petite" ?
Exemple, Civilité : M., Mme, Melle, Null

- Arbre B+ : inadéquat car chaque valeur est peu sélective
- Hachage : inadéquat car il y a beaucoup de collisions

$\Rightarrow$ Or, situation fréquente surtout lorsque les données sont agrégées comme dans les entrepôts de données (*DataWareHouse*)

7. Index Bitmap - b. Principe

Soit n le nombre d'enregistrements à indexer $\{e_1, e_2, ..., e_n\}$ et A un attribut dont le domaine de valuers comporte m valeurs possibles $\{a_1, a_2, ..., a_m\}$

1. Pour chaque valeur a_j un tableau de n bits est construit. Chaque bit du tableau représente un enregistrement
2. La i^{eme} case du tableau de a_j est mis à 1 si $e_i.A = a_j$, sinon le bit est mis à 0

7. Index Bitmap - b. Principe

	NSS	Civ.	...	Sal.
e1		Melle		
e2		Mme		
e3		M.		
e4		M.		
e5		Mme		
e6		M.		
e7		M.		
e8		M.		
e9		Melle		
e10		Null		
e11		Mme		
e12		M.		

	e1	e2	e3	e4	e5	e6	e7	e8	e9	e10	e11	e12
Melle	1	0	0	0	0	0	0	0	1	0	0	0
Mme	0	1	0	0	1	0	0	0	0	0	1	0
M.	0	0	1	1	0	1	1	1	0	0	0	1
Null	0	0	0	0	0	0	0	0	0	1	0	0

7. Index Bitmap - c. Recherche

Recherche d'une clé k

1. Consultation du bitmap de k pour identifier les enregistrements ayant cette valeur
2. Accès aux enregistrements

Recherche multi-attributs $k_1, k_2, ...k_n$

1. Application d'un opérateur logique sur les tableaux de bits associés à $k_1, k_2, ...k_n$ (ET, OU, NON, XOR, etc.)
2. Accès aux enregistrements

Calcul d'agrégat

1. Certaines requêtes peuvent être résolues sans même accéder aux données
2. Exemple :

```
SELECT COUNT(*) FROM Emp WHERE Civilité IN ('Melle', 'Mme')
```

7. Index Bitmap - c. Quand utiliser un Bitmap?

Selon Oracle :

- Petite cardinalité du domaine de valeurs : lorsque la cardinalité du domaine est 100 fois plus petite que le nombre de tuples (enregistrements)
  $\Rightarrow$ en moyenne à chaque valeur correspond 100 enregistrements
- Très peu d'insertions/mises-à-jour

Mais aussi,

- Entrepôts de données où les données sont agrégés par des attributs catégoriels (Ex. Date, Ville, Secteur, etc.) et les requêtes de calcul d'agrégats sont fréquentes (Ex. ventes/mois, ventes/région, etc.)
- Grand volume de données : occupe très peu de place comparativement à un Arbre B+

8. Filtres de bloom

- Un filtre de bloom n'est pas proprement dit une structure d'indexation, mais une structure de filtrage
- Il permet de tester rapidement
  - l'absence d'un élément ou d'une clé k dans un ensemble (avec certitude)
  - la présence d'un élément ou d'une clé k dans l'ensemble (avec une marge d'erreur - taux de faux positifs)
- Un filtre de bloom est composé de
  - T : un tableau booléen de taille m, dont les cases sont initialisées à 0.
  - $(h_i)_{1 \leq i \leq H}$: H fonctions (de hachage) de
    $$D \rightarrow [0, m-1]$$
    , où D est le domaine de valeurs de k, .
- L'ajout d'une clé k à l'ensemble se fait en appliquant les H fonctions de hachage sur k, puis à mettre à 1 les cases d'indice $h_i(k)$ (pour i variant de 0 à $H-1$) dans le tableau T .
- La vérification de l'existence d'une clé k consiste à vérifier si toutes les cases d'indices $h_i(k)$ (i variant entre 1 et k) sont à 1 dans T
- Exemple : `http://llimllib.github.io/bloomfilter-tutorial/`

8. Filtres de bloom - Faux positifs

- Le taux de faux positifs dépend de H, m et K le nombre de clés.
- Probabilité qu'un bit soit à 1 pour une seule fonction de hachage et une seule clé : $\frac{1}{m}$
- Probabilité qu'un bit soit à 0 pour une seule fonction de hachage et une seule clé : $1 - \frac{1}{m}$
- Probabilité qu'un bit soit à 0 pour H fonctions de hachage et une seule clé : $(1 - \frac{1}{m})^H$
- Probabilité qu'un bit soit à 0 pour H fonctions de hachage et K clés : $(1 - \frac{1}{m})^{H.K}$
- Probabilité qu'un bit soit à 1 pour H fonctions de hachage et K clés : $1 - (1 - \frac{1}{m})^{H.K}$
- Probabilité qu'une clé soit testée présente alors qu'elle ne l'est pas : probabilité que H positions soient à 1.

$$\left(1 - \left(1 - \frac{1}{m}\right)^{H.K}\right)^H$$

Chapitre 3 - II. Structures d'indexation textuelle en mémoire

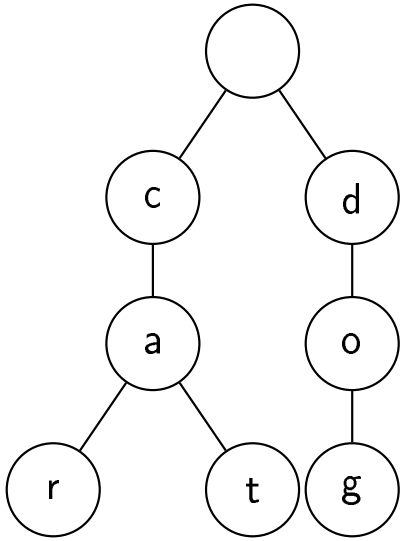
- 1. TRIE : Arbre de préfixes
- 2. FST
- 3. Exercice

1. TRIE : motivation et structure

Le **TRIE** est une structure de données arborescente optimisée pour la recherche de chaînes de caractères.

Pourquoi utiliser un Trie ?

- **Autocomplétion** : Trouver tous les mots commençant par un préfixe est immédiat (on descend dans l'arbre).
- **Recherche rapide** : Temps de recherche en $O(L)$ (où L est la longueur du mot), indépendant du nombre de mots stockés.
- **Indexation** : Sert de dictionnaire pour mapper un **terme** vers sa **posting list**.



Exemple : car, cat, dog

Structure : Les clés ne sont pas stockées dans les nœuds, mais définies par le chemin depuis la racine.

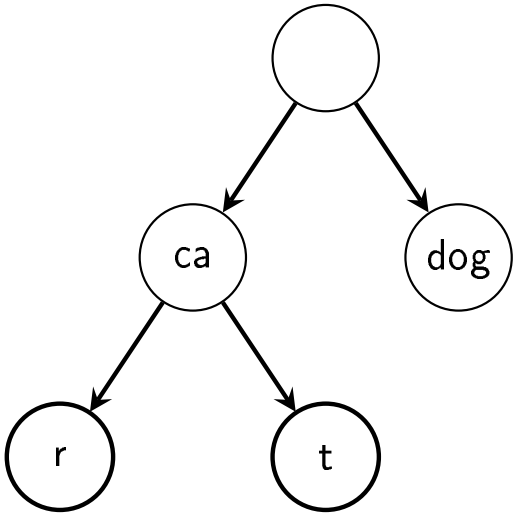
1. Optimisation : Radix Trie (ou Patricia Tree)

Le problème du Trie classique : Beaucoup de nœuds "inutiles" (un seul enfant) qui consomment de la mémoire.

L'idée du Radix Trie : On fusionne les nœuds parents avec leurs enfants uniques.

- On réduit le nombre de nœuds.
- Les étiquettes deviennent des **chaînes de caractères**.
- Moins de "sauts" en mémoire lors de la recherche.

Version Compactée



Gain : On passe de 7 nœuds à 5 nœuds. Les préfixes sont toujours partagés.

1. Limite du Trie

Le problème : On veut stocker :

- BAS → 0
- CAS → 1
- PAS → 2

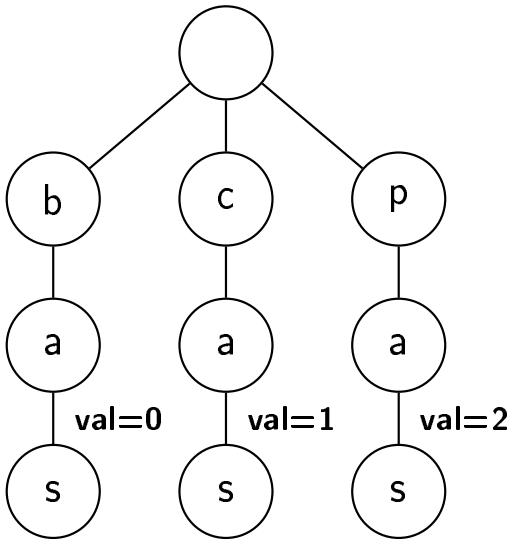
• **Unicité des feuilles :** Puisque bas → *0* et cas → *1*, les nœuds terminaux ne peuvent pas être fusionnés, car ils pointent vers des données différentes.

• **Obstacle à la compression :** Même si des milliers de mots partagent le même suffixe (ex: *-as*), le Trie doit maintenir des branches séparées jusqu'à la fin.

• **Explosion mémoire :**

- 1 mot = 1 nœud feuille dédié.
- Pour un dictionnaire de 10 millions de termes, on obtient **10 millions de nœuds feuilles**.

Structure en Arbre (Trie)



9 nœuds nécessaires.

Chapitre 3 - II. Structures d'indexation textuelle en mémoire

- 1. TRIE : Arbre de préfixes
- 2. FST
- 3. Exercice

2. Idée du FST

FST = Finite State Transducer (Fast Succinct Trie)

Comme un trie MAIS :

- Les transitions produisent une sortie
- Les sorties s'additionnent le long du chemin
- Les suffixes identiques peuvent être fusionnés

Résultat :

- Structure minimale
- Très compacte

2. Rôle du FST dans un moteur de recherche

Organisation classique d'un index :

- Les termes sont stockés **triés lexicographiquement**
- Les postings lists sont stockées **sur disque**
- Chaque terme est associé à :
  - un **ordinal**
  - ou un **offset disque**

Important :

- Le FST ne stocke **pas** les postings
- Il stocke une **valeur numérique compacte**

À la fin du parcours :

valeur accumulée $\Rightarrow$ position de la posting list

Exemple simplifié :

Terme	Ordinal
car	0
cat	1
dog	2

Sur disque :

Posting 0 $\rightarrow$ car
Posting 1 $\rightarrow$ cat
Posting 2 $\rightarrow$ dog

2. Construction d'un FST (exemple complet)

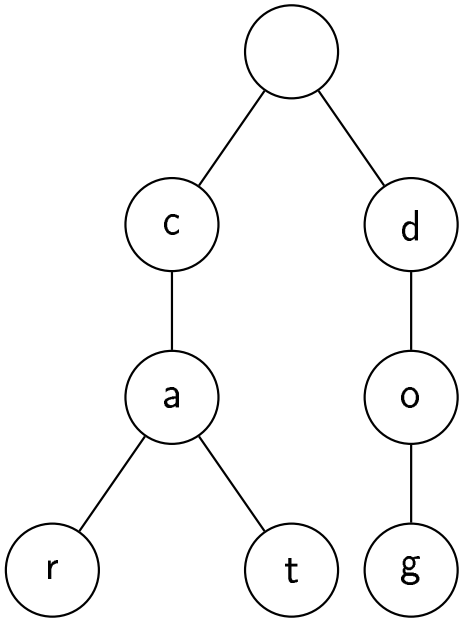
Objectif :

Terme	Ordinal
car	0
cat	1
dog	2

Les mots doivent être triés :

car cat dog

Étape 1 : construire le trie



2. Redistribution des sorties : principe

Idée clé : Un FST calcule une fonction

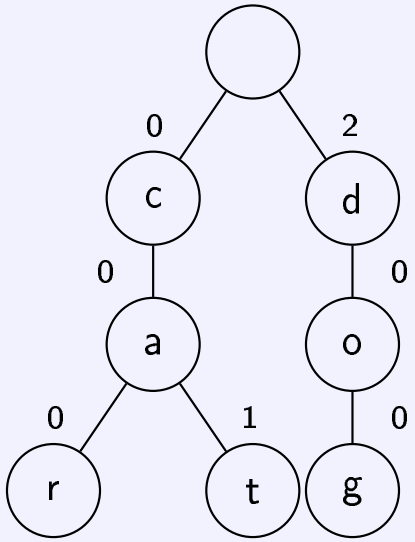
$$f(\text{mot}) = \sum \text{sorties le long du chemin}$$

Propriété fondamentale :
On peut redistribuer les valeurs sur les arcs tant que la somme finale reste identique.

Pourquoi cela marche ?

- Les mots sont triés
- Les ordinals sont croissants
- Les différences sont petites

On encode donc des **différences** (delta) au lieu des valeurs complètes.



Calcul :

car $\rightarrow$ 0+0+0 = 0
cat $\rightarrow$ 0+0+1 = 1
dog $\rightarrow$ 2+0+0 = 2

2. Étape 2 : Minimisation (Fusion des suffixes)

Le secret de la compression :

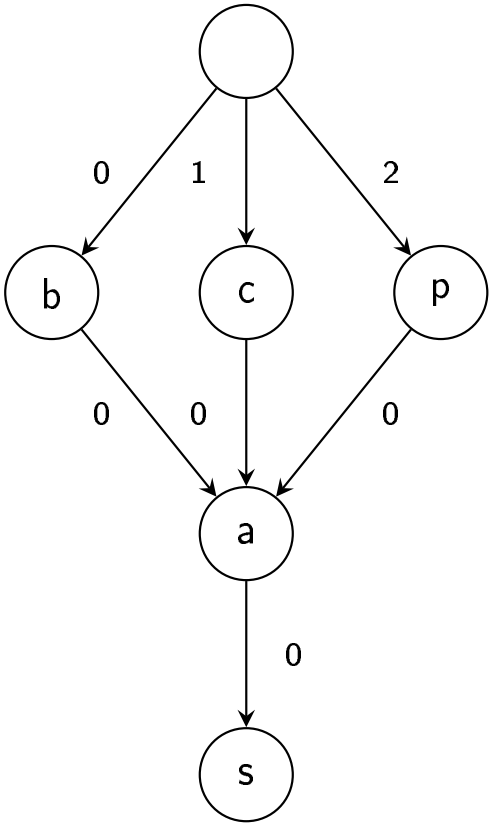
Une fois les sorties "poussées" vers la racine, les chemins terminaux deviennent identiques.

- **Trie** : On ne peut pas fusionner les branches car les feuilles portent des valeurs différentes (0, 1, 2).
- **FST** : Les valeurs étant portées par les premières lettres, le suffixe **-AS** est strictement identique pour les trois mots.

Gain mémoire

On fusionne les états redondants. L'arbre devient un **graphe orienté acyclique** (DAG).

Exemple : BAS, CAS, PAS



Suffixe **-as** fusionné en un seul chemin.

2. Comparaison : Trie vs FST

Propriété	**Trie classique**	**FST (Lucene)**
Partage de préfixes	Oui	Oui
Partage de suffixes	**Non**	**Oui**
Sorties	Sur les nœuds finaux	Sur les transitions
Utilisation mémoire	Élevée (Pointeurs)	Très faible (Tableaux compressés)
Recherche $O(length)$	Oui	Oui

Pour 1 million de termes, un Trie peut prendre plusieurs Go, là où un FST tiendra en quelques dizaines de Mo.

2. L'encodage "Succinct"

- **Trie classique (Objet)** : Chaque nœud est un objet en mémoire. Un pointeur vers un fils occupe **8 octets**. Pour 1 million de nœuds, la gestion des pointeurs consomme énormément.

L'approche FST (Lucene)

Le FST n'est pas un graphe d'objets, mais un **tableau d'octets continu** (*byte array*).

- **Pas de pointeurs mémoire** : On utilise des **adresses relatives** (offsets) dans le tableau.
- **Variable-Length Encoding** : Si un saut est court, il est encodé sur 1 seul octet.
- **Bit-Packing** : On utilise uniquement le nombre de bits nécessaires pour stocker les sorties numériques.

Résultat : Le FST est si petit qu'il tient entièrement dans le **cache L3** du processeur, ce qui rend la traversée extrêmement rapide par rapport à un Trie fragmenté en RAM.

2. L'encodage binaire (Byte Array)

FST : pas un graphe d'objets liés par des pointeurs, mais un **tableau d'octets**.

Vue conceptuelle du tableau mémoire :

Position	Lettre	Sortie	Cible (Saut)	
0	c	0	**vers pos. 2**	← Racine
1	d	2	**vers pos. 5**	
2	a	0	**vers pos. 3**	← État après 'c'
3	r	0	**FIN**	← car = 0
4	t	1	**FIN**	← cat = 1
5	o	0	...	

Pourquoi est-ce plus compact ?

- **Pas de pointeurs 64 bits :** On stocke juste un petit nombre (l'index dans le tableau).
- **Localité :** Le CPU lit le tableau d'un bloc, sans chercher des objets éparpillés en RAM.

2. Pourquoi Lucene utilise un FST

Dans **Apache Lucene** :

- Les termes sont triés lexicographiquement
- Le FST mappe terme → ordinal
- L'ordinal donne accès aux postings

Pourquoi pas un B+Tree ?

- FST est en mémoire
- Plus compact
- Plus rapide

Chapitre 3 - II. Structures d'indexation textuelle en mémoire

- 1. TRIE : Arbre de préfixes
- 2. FST
- 3. Exercice

3. Exercice

On considère les mots suivants (triés) :

`car` `cat` `dog`

On veut construire un FST minimal qui associe :

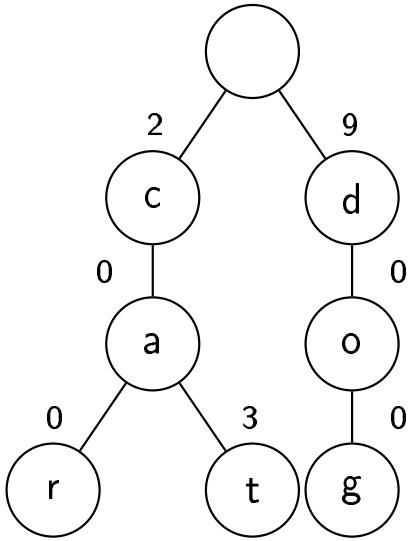
car $\rightarrow$ 2
cat $\rightarrow$ 5
dog $\rightarrow$ 9

Dessiner le FST correspondant.

3. Exercice

1. **Calcul des deltas :**
   - car (2) et cat (5) partagent le préfixe **"ca"**.
   - On place le minimum sur l'arc **'c'**.
   - **Arc 'c'** $\leftarrow 2$
   - **Arc 'r'** $\leftarrow 2 - 2 = 0$
   - **Arc 't'** $\leftarrow 5 - 2 = 3$
   - **Arc 'd'** $\leftarrow 9$ (pas de partage)

FST Minimal



2. **Vérification pour** cat **:**

Chemin : $c(2) \rightarrow a(0) \rightarrow t(3)$

$$2 + 0 + 3 = 5$$